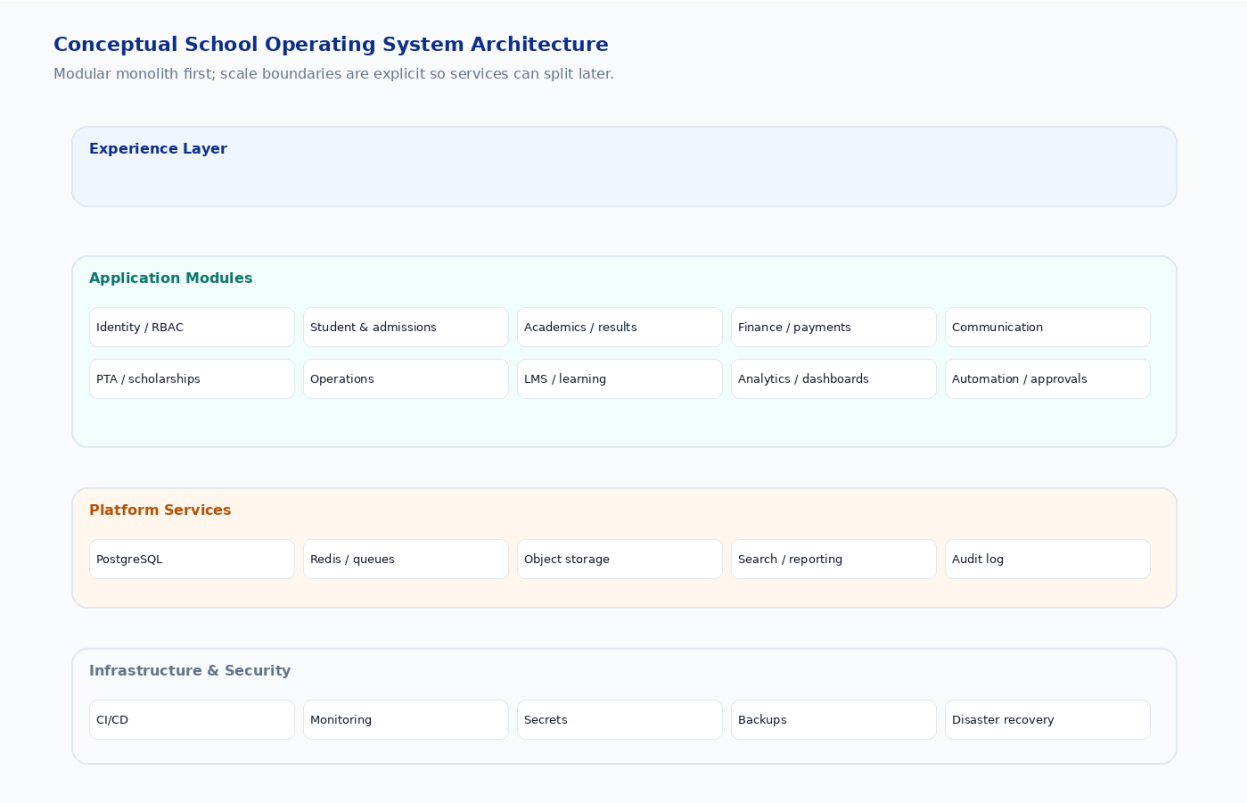
SCHOOL OPERATING SYSTEM

Complete Product Validation, Feature Blueprint, Security Architecture and Development Playbook



Conceptual architecture visual. Final product branding and infrastructure choices remain configurable.

Prepared as a master blueprint for product discovery, engineering, school deployment and handover

Important scope note

This book is a product and engineering blueprint. It is not legal advice, a security certification, a PCI certification, or a guarantee of regulatory compliance. The final production system must be reviewed against the school contract, current Nigerian law, vendor terms, and an appropriate security/compliance assessment.

Version: 1.0 | Date: 28 September 2026

Working product name: SchoolOS (placeholder)

Primary market assumption: Nigerian nursery/primary/secondary schools, with architecture suitable for wider markets.

Contents

1. 1. Executive summary and idea validation
2. 2. Product vision and operating model
3. 3. Market reality and differentiation
4. 4. Product principles and non-negotiable standards
5. 5. Users, roles and organizational structure
6. 6. Complete feature map
7. 7. School administration and setup
8. 8. Admissions and enrollment
9. 9. Student information system
10. 10. Parent, teacher and student portals
11. 11. Academics, timetable and attendance
12. 12. Assessments, exams, results and report cards
13. 13. Fees, payments and finance
14. 14. Communication and notification system
15. 15. PTA, events and community
16. 16. Scholarships, sponsorships and vouchers
17. 17. Library, clinic, transport, hostel, cafeteria and assets
18. 18. Learning management system
19. 19. AI and intelligent automation
20. 20. Dashboards, analytics and visual reporting
21. 21. Approval workflows and separation of duties
22. 22. Security architecture
23. 23. Privacy, data governance and Nigerian compliance
24. 24. Backup, disaster recovery and business continuity
25. 25. Scalability and multi-tenant architecture
26. 26. Data model and database design
27. 27. API, integrations and background jobs
28. 28. Development standards and testing
29. 29. Antigravity development workflow
30. 30. Step-by-step development roadmap
31. 31. Deployment, release and operations
32. 32. School implementation and data migration
33. 33. School IT handover model
34. 34. Support, maintenance and service operations
35. 35. Pilot validation and product acceptance
36. 36. Commercial packaging and school proposal strategy
37. 37. KPI framework
38. 38. Risk register and mitigation
39. 39. Master screen/page inventory
40. 40. Approval matrix
41. 41. Definition of done and release checklist
42. 42. Reference architecture summary

43. 43. Reference sources and compliance notes

Navigation note: each numbered chapter is intended to be treated as an independently reviewable product area. Antigravity tasks should reference these chapters and the corresponding acceptance criteria rather than asking an agent to build the entire platform in one step.

1. Executive summary and idea validation

The proposed product is a multi-tenant School Operating System: a secure SaaS platform that unifies school administration, student records, academics, attendance, fees, online payments, communication, parent engagement, school operations, scholarships, reporting and eventually online learning. It is intentionally broader than a basic school management dashboard because the product is designed around connected workflows rather than isolated features.

What the idea is solving

Schools often maintain multiple sources of truth: paper files, spreadsheets, messaging groups, separate result tools, bank alerts, payment portals, websites and learning platforms. The product thesis is that the operational cost is created not only by the individual tasks but by reconciliation between systems. A connected student record, permission model and workflow engine can reduce duplicate entry, automate routine actions and give each stakeholder a role-specific view.

Validation conclusion

The category is commercially real and already competitive in Nigeria. Current vendors advertise broad coverage including admissions, attendance, fees, results, parent communication, operations and learning. That means the existence of demand is plausible, but feature count alone is not a sufficient differentiator. The product must validate a specific wedge - for example stronger governance, approval workflows, data ownership, implementation quality, recovery capability, or a school-specific operating experience. See Chapter 3.

Validation evidence reviewed

Evidence	What it shows	How to use it
Edves	Vendor states it serves 2,300+ schools and covers management, finance, assessment, AI, parent engagement and operations.	Treat as evidence of category activity and competitive breadth; vendor claims are not independently verified.
SchoolTry	Vendor advertises K-12 management, virtual classroom, finance, CBT, HR, hostel, document management, dashboards and LMS.	Confirms schools can expect broad functional scope from established vendors.
NoviaSMS	Vendor advertises school records, fees, results, attendance and a second-person approval flow for sensitive changes.	Validates that maker-checker governance is a commercially understandable feature, not merely an engineering concept.
NDPC	Nigeria has a statutory data protection framework and current implementation guidance.	Treat privacy, security, data subject rights and processor/controller responsibilities as product requirements.
OWASP / NIST / CISA	Current security and recovery guidance emphasizes authentication, authorization, logging, backup integrity, recovery planning and testing.	Use as engineering baselines and verification inputs, not as a certification claim.

Validation plan before mass rollout

44. Interview 5-10 schools across different sizes and operating models; document the current process rather than pitching features first.
45. Observe how admissions, fees, attendance, results and parent communication are actually handled.
46. Create a clickable prototype of the end-to-end student lifecycle and dashboard experience.
47. Run demonstrations using a school-specific sample dataset and fee structure.
48. Secure at least one pilot agreement with written success criteria, data responsibilities and support rules.
49. Run the platform in a controlled pilot, ideally with a parallel period where the existing process remains available.
50. Measure adoption, time saved, reconciliation errors, payment collection workflow, support volume and user satisfaction.
51. Only after the pilot passes should the product packaging and broader school sales process be scaled.

Do not confuse validation with coding

The first successful school deployment is stronger evidence than a large feature list. Build the system to commercial quality, but validate the actual school workflow continuously while developing it.

2. Product vision and operating model

Vision

Give a school one secure operational system from admission to graduation, with parents and students able to access the information they need, and with school leadership able to see the health of the institution through live, permission-aware dashboards.

Core promise

One student record. One operational source of truth. Controlled workflows. Actionable dashboards. Secure payments. Recoverable data. Clear accountability.

Operating model

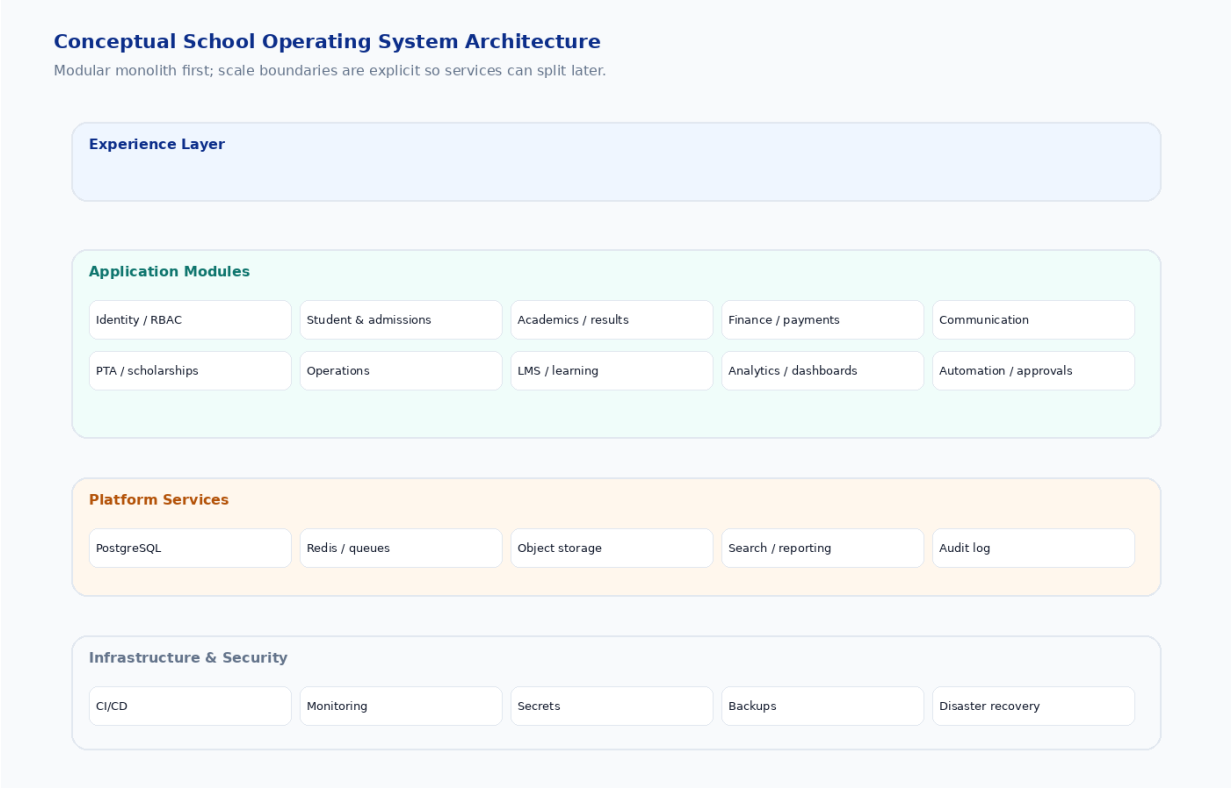


Figure 1. Product architecture at a conceptual level.

Product layers

Layer	Purpose	Examples
Experience	Role-specific user interfaces	Owner, principal, teacher, parent, student, IT, public website
Domain modules	School functions	Admissions, student records, academics, finance, PTA, transport, LMS
Workflow	Automated and approval-driven processes	Result approval, fee reminders, data corrections, refunds

Data	Persistent institutional record	Students, guardians, classes, payments, results, documents
Platform	Shared technical capabilities	Identity, audit log, storage, notifications, jobs, search
Security / operations	Protection and continuity	MFA, encryption, backups, monitoring, incident response

Source of truth principle

If a payment is recorded, every authorized view that depends on that payment should derive from the same authoritative transaction record. If a student changes class, the class membership, timetable visibility and reporting views should update from that record. Avoid duplicating mutable facts across separate tables or services unless a deliberate read model, cache or ledger design explains the duplication.

3. Market reality and differentiation

The Nigerian market already has serious school-management products. Edves markets integrated management, finance, assessment, curriculum and parent engagement; SchoolTry markets broad school administration, virtual classroom, CBT, finance, HR, document management, dashboards and LMS; and other vendors market similar combinations. This means the product should not be positioned simply as "another school management system."

Potential differentiation to test

- School governance by design: sensitive actions require approval by the right people and every change is auditable.
- Operational resilience: backup, restore testing, offline-tolerant workflows where practical, and clear disaster recovery procedures.
- School IT handover: the customer receives a documented, controllable operating model rather than becoming dependent on one developer.
- Data transparency: dashboards show the underlying record, timestamp, filters and drill-down path so leadership can understand where every number came from.
- Implementation quality: migration, configuration, training, parallel run, acceptance and handover are treated as first-class product services.
- Flexible architecture: schools can use external providers for payments, messaging or video without losing the central school record.
- Configurable governance: schools can define who can request, approve, execute and reverse sensitive actions.

Competitive reality table

Market expectation	Your response
Admissions and student management	Implement deeply and make the student lifecycle consistent across modules.
Fees and online payment	Treat the payment ledger, reconciliation and approval workflow as a core subsystem.
Results and report cards	Build a controlled score-to-report pipeline with locking and approval.
Parent communication	Unify in-app, email, SMS and WhatsApp integration while keeping the platform as the source of truth.
LMS / virtual classroom	Integrate external video platforms initially; develop native classroom capabilities later only if justified.
Dashboards	Make dashboards role-specific, actionable, filterable and traceable to underlying records.

Strategic implication

Do not compete by adding every possible feature faster than established vendors. Compete on reliability, governance, implementation quality, user experience, and the ability to prove where every important number and change came from.

4. Product principles and non-negotiable standards

Principle	Requirement
Security by design	Threat modeling, secure defaults, least privilege and protected secrets are part of feature design.
Privacy by design	Collect only what is necessary, document purpose, control access, support retention and rights workflows.
Tenant isolation	Every school is logically isolated. Cross-school access is denied by design and tested explicitly.
Least privilege	Users see and change only what their role and assignment permits.
Separation of duties	Sensitive actions can require different people to request, approve and execute.
Traceability	Sensitive changes have who/what/when/why/before/after context.
Recoverability	Backups are isolated where necessary, monitored and restored in tests.
Idempotency	Financial and asynchronous operations are safe to retry without duplicating effects.
Human override	Automation can be reviewed and corrected by authorized people; irreversible automated actions are minimized.
Observable dashboards	Metrics include definitions, timestamps and drill-down paths.
Configurable school model	Different schools can use different grading, fee, class and workflow settings without separate codebases.
Accessible UX	Important tasks should remain usable on common phones, tablets and desktop devices.

Quality gates

52. Product acceptance criteria are written before implementation.
53. Test data covers positive, negative, boundary and permission scenarios.
54. No feature is considered complete if authorization is not tested.
55. Any migration changes are reversible or have an explicit recovery strategy.
56. Every production release has monitoring, rollback and migration notes.
57. Any sensitive feature has an audit-event specification.
58. Any data-destructive capability has explicit approval and recovery controls.

5. Users, roles and organizational structure

Multi-level organization

Level	Typical roles	Main authority
Platform	SaaS owner, platform admin, support engineer, security/operator	Tenant lifecycle, billing, platform operations, support, incidents
School ownership	Proprietor, school owner, board delegate	Business policy, high-risk approvals, school configuration
School management	Principal, vice principal, registrar, bursar, academic coordinator	Daily school operations
Operational staff	Teachers, class teachers, librarian, nurse, transport, hostel	Assigned functional tasks
Family	Parent/guardian	View own children, payments, requests, communications
Student	Student	Own learning, timetable, results and permitted student functions

Role design rule

Roles should not be hard-coded only as a single enum. Use roles, permissions, assignments and resource scope. For example, a teacher may have "enter attendance" permission only for classes assigned to that teacher. A bursar may have "view payments" but not "change academic results." A school IT administrator may manage user accounts but should not automatically receive permission to alter protected academic or financial records.

Example permission structure

Permission	Scope example	Approval
students.read	All school students or assigned campus	None
students.edit_basic	Assigned students	Level 1 or configured
students.edit_identity	Protected identity fields	Level 2
results.enter	Assigned classes/subjects	None
results.edit_locked	Locked result record	Level 2
payments.record	Finance module	None
payments.reverse	Specific payment	Level 2
roles.assign_privileged	Privileged role	Level 2 or 3
data.export_sensitive	Sensitive student dataset	Level 2
production_restore	Production recovery	Level 3

6. Complete feature map

The following modules form the target product scope. The implementation roadmap later in this book separates core commercial launch from later expansion; the complete product can still be specified now so architecture decisions are made with the destination in mind.

Administration

- School profile
- Campuses / branches
- Academic sessions
- Terms
- Class structure
- Subjects
- Departments
- School calendar
- Announcements
- Documents
- Configuration
- Branding
- Working hours
- Holidays
- Custom fields

Admissions

- Inquiry capture
- Application forms
- Document upload
- Application fee
- Entrance test
- Interview scheduling
- Scoring
- Review workflow
- Offer letter
- Acceptance
- Waitlist
- Rejection
- Application analytics
- Applicant-to-student conversion

Students

- Student profile

- Admission number
- Guardian links
- Emergency contacts
- Medical fields
- Documents
- Class history
- House / arm
- Behaviour
- Certificates
- Transfer / withdrawal
- Alumni record
- Student search
- Bulk import

Parents

- Multiple children
- Family profile
- Child switcher
- Fees
- Payments
- Receipts
- Results
- Attendance
- Assignments
- Messages
- Notices
- Events
- Appointments
- Requests
- Permission forms

Teachers

- Teacher profile
- Subjects
- Classes
- Timetable
- Attendance
- Assignments
- Assessments
- Score entry
- Result submission
- Learning resources
- Parent communication

- Announcements
- Workload

Academics

- Curriculum setup
- Subject allocation
- Timetable
- Lesson plans
- Attendance
- Assignments
- Assessments
- Exams
- Grade rules
- Results
- Promotion
- Academic analytics

Finance

- Fee structures
- Invoices
- Invoice items
- Installments
- Discounts
- Scholarships
- Online payments
- Payment verification
- Receipts
- Refunds
- Reversals
- Expenses
- Budgeting
- Reports
- Reconciliation

Communication

- In-app notifications
- Email
- SMS
- WhatsApp integration
- Push notifications
- Announcements
- Message threads

- Templates
- Broadcasts
- Emergency alerts
- Delivery status

PTA

- Membership
- Meetings
- Attendance
- Notices
- Projects
- Contributions
- Financial reports
- Polls / votes
- Documents
- Event registration

Scholarships

- Scholarship definitions
- Eligibility
- Applications
- Review
- Approval
- Awards
- Renewals
- Sponsor records
- Vouchers
- Redemption history

Operations

- Library
- Clinic
- Transport
- Hostel
- Cafeteria
- Inventory
- Assets
- Maintenance
- Visitors
- Incidents
- School events

LMS

- Courses
- Modules
- Lessons
- Resources
- Video
- Quizzes
- Assignments
- Submissions
- Progress
- Certificates
- Live class links
- Discussion

AI / automation

- Question generation
- Lesson assistance
- Performance summaries
- Fee follow-up automation
- Attendance alerts
- Workflow engine
- Scheduled reports
- Anomaly alerts
- Natural-language reporting
- Admin assistant

Platform

- Multi-tenancy
- Subscriptions
- Billing
- Usage metering
- Feature flags
- Support tickets
- Audit logs
- Security events
- Backups
- Monitoring
- Integrations
- API keys
- Data export

7. School administration and setup

School creation wizard

59. Create organization
60. Verify owner account
61. Configure school identity
62. Create campus(es)
63. Create academic session
64. Create terms
65. Configure school levels
66. Create class and arm structure
67. Import staff
68. Import students and guardians
69. Configure fee structure
70. Configure grading rules
71. Configure notifications
72. Invite users
73. Run readiness checklist

School configuration must be data-driven

Do not hard-code "JSS 1" or a fixed three-term model into application logic. Store educational structures as configurable records. The same platform should support a school with one campus and another with multiple campuses, provided the tenancy and scope rules remain correct.

Configuration areas

Area	Examples
Academic	Session dates, terms, periods, class levels, subjects, grading scale
Finance	Fee types, billing cycles, discounts, installment rules, payment gateways
Attendance	Statuses, lateness thresholds, escalation rules
Results	CA weights, exam weights, grade bands, approval stages
Communication	Templates, channels, scheduled notices, emergency contacts
Governance	Approval thresholds, privileged roles, dual-control rules
Brand	Logo, colors, report header, receipts, portal identity

8. Admissions and enrollment

End-to-end workflow

74. Prospective parent starts application
75. Application form validates fields
76. Documents are uploaded and scanned
77. Application fee is generated if applicable
78. Entrance test / interview is scheduled
79. Reviewers score and comment
80. Authorized reviewer approves or rejects
81. Offer is generated
82. Parent accepts offer
83. Deposit / registration fee is paid if applicable
84. Applicant is converted into a student
85. Guardian relationships and class assignment are created
86. Welcome / onboarding tasks are triggered

Data quality controls

- Duplicate applicant detection
- Phone/email normalization
- Required-document checks
- Age/date validation
- Class capacity rules
- Payment reference uniqueness
- Manual review for unusual records
- Complete audit trail for status changes

Admission analytics

- Applications by source
- Conversion rate
- Applications by class
- Pending reviews
- Test performance
- Interview outcomes
- Offers accepted
- Deposit collection
- Drop-off points

9. Student information system

The student record is the connective tissue of the platform. It should expose stable identifiers and historical relationships without overwriting history.

Student record sections

- Identity and biodata
- Guardians
- Emergency contacts
- Admission history
- Current enrollment
- Class history
- Attendance
- Assessment and results
- Fees and transactions
- Documents
- Behaviour and welfare
- Library
- Transport
- Hostel
- Clinic
- Certificates
- Alumni / exit status

Historical integrity

Avoid destructive updates where a historical record matters. A class transfer should create a new enrollment period, not rewrite the old one. A corrected guardian relationship should preserve an audit record. A withdrawn student should remain in historical reports according to the school retention policy rather than disappearing from the system.

Search and discovery

- Search by admission number
- Name and guardian
- Phone/email where permitted
- Class and arm
- Session
- Payment status
- Attendance risk
- Result status
- Document status

10. Parent, teacher and student portals

Parent portal

- Dashboard
- Child switcher
- Fees and invoices
- Payments and receipts
- Results
- Attendance
- Assignments
- Messages
- Announcements
- Events
- PTA
- Transport
- Documents
- Requests
- Appointments
- Notifications

Teacher portal

- Dashboard
- Today view
- Assigned classes
- Timetable
- Attendance
- Assignments
- Assessments
- Score entry
- Result submission
- Learning materials
- Parent communication
- Announcements
- Class performance

Student portal

- Dashboard
- Timetable
- Subjects
- Lessons
- Assignments

- Quizzes
- Results
- Attendance
- Learning resources
- Certificates
- Announcements
- Messages

Child account design

Primary-school users may have restricted student accounts or parent-controlled access. The platform should support configurable age/role policies rather than assuming the same access model for every school.

11. Academics, timetable and attendance

Timetable engine

- Class timetable
- Teacher timetable
- Room timetable
- Period rules
- Breaks
- Subject frequency
- Teacher availability
- Room capacity
- Conflict detection
- Printable timetable
- Role-specific timetable view

Attendance engine

Attendance should be more than a register. The system calculates daily, weekly, monthly and term-level indicators and feeds configurable alerts. Every absence or edit should have an identifiable actor and timestamp.

Scenario	Automatic action
Student marked absent	Update attendance aggregate; apply school rule for parent notification.
Repeated absence threshold reached	Create attention item for class teacher / administrator.
Late threshold exceeded	Record lateness and update attendance analytics.
Attendance edited after lock	Require configured approval and record before/after values.
Term closes	Freeze period according to school policy; preserve historical view.

Offline tolerance

For schools with intermittent connectivity, attendance can use an offline-capable pattern where safe: locally queue a limited transaction set, timestamp it, and synchronize when connectivity returns. Conflict rules and secure local storage are mandatory; offline is not a license to bypass authorization.

12. Assessments, exams, results and report cards

Result pipeline

87. Configure assessment structure
88. Create assessments
89. Assign students / classes
90. Teacher enters or imports scores
91. Validate score ranges
92. Detect missing scores
93. Submit for review
94. Moderate if required
95. Approve
96. Lock
97. Generate report
98. Publish to parent/student
99. Record corrections only through controlled workflow

Result correctness rules

- Prevent impossible scores
- Prevent duplicate assessment entries
- Validate subject/class membership
- Show missing scores explicitly
- Store grading scheme version
- Record who submitted and approved
- Lock published results
- Require reason for post-lock correction

Report-card generation

- School-branded template
- Academic summary
- Subject results
- Attendance
- Teacher comments
- Principal / head comment
- Affective / psychomotor fields where configured
- Fee notice if school wants it
- Digital verification code
- PDF output
- Archive copy

Governance rule: Locked result changes require a controlled request with before/after values, reason, requester, approver and audit trail.

13. Fees, payments and finance

Financial subsystems

- Fee structure
- Student charges
- Invoices
- Installment plans
- Discounts
- Scholarships
- Payment intents
- Payment gateway callbacks
- Reconciliation
- Receipts
- Refunds
- Reversals
- Expenses
- Budget
- Financial reports
- Export

Payment lifecycle

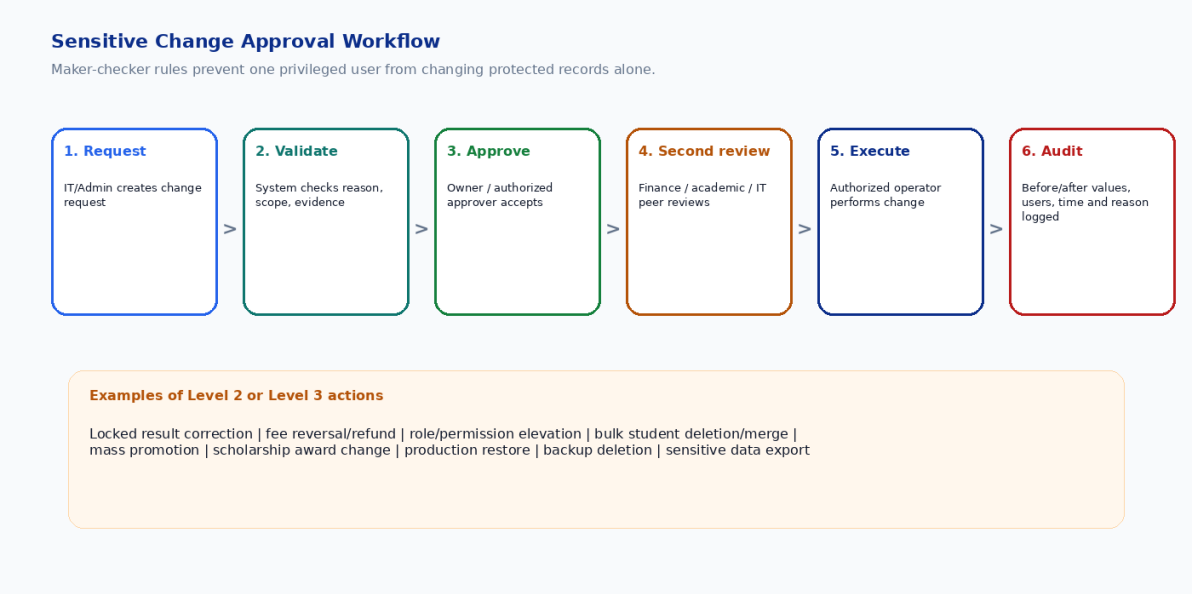


Figure 2. Sensitive-change workflow; the same governance pattern should be used around refunds and payment reversals.

For online payments, the platform should create a payment record and reference, redirect or invoke the provider as appropriate, verify callbacks server-side, enforce idempotency, reconcile status, and record the final state. The application should not trust a browser redirect alone as evidence that money was received.

Payment provider boundary

Use a compliant payment provider to handle card and other payment methods rather than storing raw card credentials in your own database. The current PCI DSS v4.0.1 release is the applicable PCI standard family for payment-card environments; the exact obligations depend on how the platform and its providers handle payment data and should be confirmed for the deployed integration. [R5]

Finance dashboard

Widget	Purpose
Billed vs collected	Shows expected amount against settled amount.
Outstanding ageing	Groups balances by age bucket.
Daily collection	Shows transaction volume and amount by day.
Payment reconciliation	Highlights unmatched or pending records.
Refund / reversal queue	Shows transactions awaiting controlled action.
Collection by class	Allows leadership to investigate collection patterns without opening every student record.

14. Communication and notification system

Notification architecture

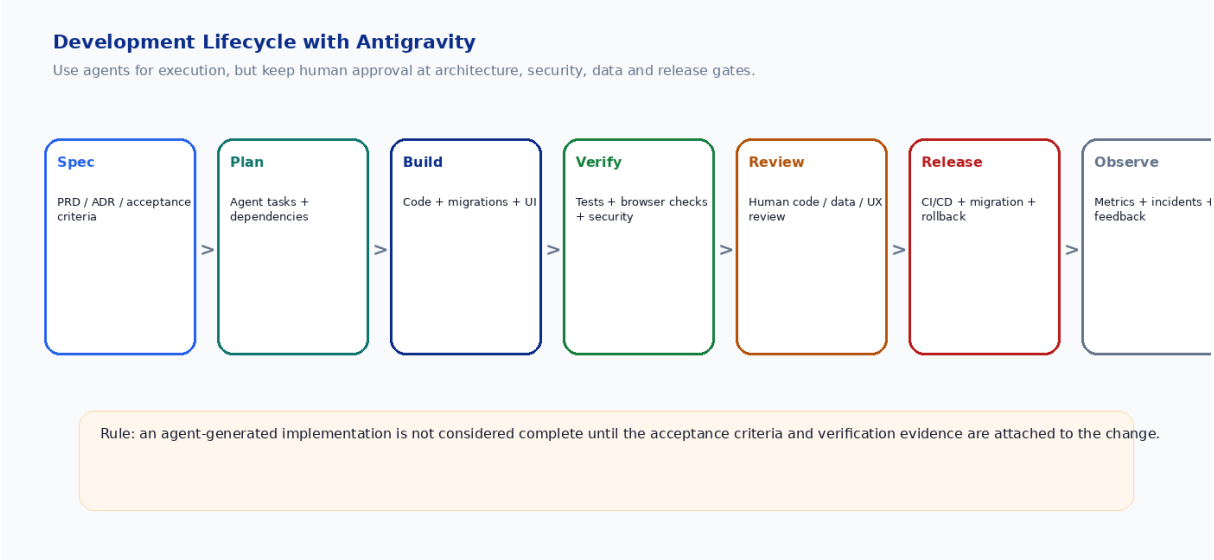


Figure 3. Agentic development lifecycle; the product notification architecture should be similarly event-driven, with clear state transitions.

Treat notifications as a reusable service. Business modules create notification events, then a notification engine decides which channel, template and audience apply. This prevents individual modules from embedding SMS, email or WhatsApp code everywhere.

Event	Possible channels
Fee due soon	In-app + email + SMS/WhatsApp
Result published	In-app + push/email
Attendance alert	In-app + configured parent channel
Emergency notice	In-app + high-priority external channel
PTA meeting reminder	In-app + email/SMS
Password reset	Email/SMS
Support ticket update	In-app + email

Delivery state

- Queued
- Sent
- Delivered where provider supports it
- Failed
- Retried
- Suppressed
- Cancelled

15. PTA, events and community

PTA

- Membership
- Meetings
- Attendance
- Agenda
- Minutes
- Announcements
- Projects
- Contributions
- Financial summaries
- Polls / votes
- Documents
- Event registration

Events

- Create event
- Audience
- Capacity
- Registration
- Fees
- Attendance
- Calendar
- Reminders
- Event reports
- Post-event documents

Community workflow

A school should be able to create a PTA initiative, define its goal, track authorized contributions and present a clear financial summary. The system should separate a contribution record from a general donation narrative so that reconciliation remains possible.

16. Scholarships, sponsorships and vouchers

Scholarship engine

100. Create scholarship programme
101. Define eligibility criteria
102. Set award type
103. Open application
104. Collect application data
105. Review and score
106. Approve
107. Generate award record
108. Apply award against eligible fees
109. Track renewal
110. Record sponsor contribution where applicable

Voucher engine

- Unique voucher code
- Value / percentage
- Eligible fee types
- Maximum redemptions
- Student restriction
- Expiry
- Activation
- Redemption audit
- Cancellation
- Fraud/rate-limit controls

Future marketplace option

A later version can connect verified sponsors with school-managed scholarship opportunities. Keep this as a separate bounded domain so it does not complicate the core school ledger prematurely.

17. Library, clinic, transport, hostel, cafeteria and assets

Library

- Catalog
- Copies
- Borrow/return
- Reservations
- Fines
- Lost items
- Search
- Reports

Clinic / welfare

- Restricted medical profile
- Emergency contacts
- Clinic visits
- Allergies
- Incident log
- Parent notification
- Access logging
- Retention controls

Transport

- Vehicles
- Drivers
- Routes
- Stops
- Student assignment
- Manifests
- Transport fees
- Maintenance
- Documents
- Optional GPS integration

Hostel

- Hostels
- Rooms
- Beds

- Allocation
- House staff
- Movement
- Check-in/out
- Incidents
- Fees

Cafeteria

- Meal plan
- Feeding status
- Dietary requirements
- Kitchen inventory
- Vendors
- Charges

Assets and maintenance

- Assets
- Location
- Condition
- Custodian
- Maintenance
- Warranty
- Disposal
- Audit

18. Learning management system

The LMS should be integrated into the school record but remain a distinct learning domain. This keeps the product coherent and avoids forcing live-video infrastructure into the first release.

Core LMS

- Courses
- Modules
- Lessons
- Video
- PDFs
- Slides
- Quizzes
- Assignments
- Submissions
- Grades
- Progress
- Certificates
- Discussion
- Learning paths

Live classes

Start with external integrations such as Zoom or Google Meet where appropriate. Store the lesson, attendance and meeting reference in the school system, while the external provider handles the video-conference infrastructure. A native classroom can be introduced later if the commercial need and technical capacity justify it.

Learning analytics

- Course completion
- Lesson engagement
- Quiz attempts
- Assignment completion
- Assessment performance
- Students needing support
- Teacher course activity

19. AI and intelligent automation

AI use cases

User	Example assistance	Control
Teacher	Generate question drafts or lesson outline	Teacher reviews before publishing
Student	Explain a concept or generate practice questions	Age-appropriate policy; no unauthorized private data access
Parent	Summarize child performance	Only data linked to the parent-authorized child
Principal	Summarize attendance/academic trends	Show supporting metrics and timestamp
Bursar	Explain fee exceptions	Use ledger data; never invent a transaction
IT admin	Summarize security events	Do not reveal secrets or raw sensitive data unnecessarily

AI governance

- AI output must be reviewable for high-impact actions
- Do not silently use school data for model training without an appropriate lawful/contractual basis and school policy
- Log AI-assisted actions where they change official records
- Keep prompts and outputs within appropriate retention rules
- Use retrieval from authorized school records rather than guessing
- Provide a clear human correction path

AI is an assistant, not the authority

Academic decisions, financial adjustments, identity changes and access-control changes should remain subject to explicit human governance. AI can prepare, summarize and flag; it should not silently make high-impact decisions.

20. Dashboards, analytics and visual reporting

The platform must present data visually, not only as tables. Every major dashboard should answer three questions: what is happening, where is it happening, and what action should the user take next.

Owner / proprietor dashboard

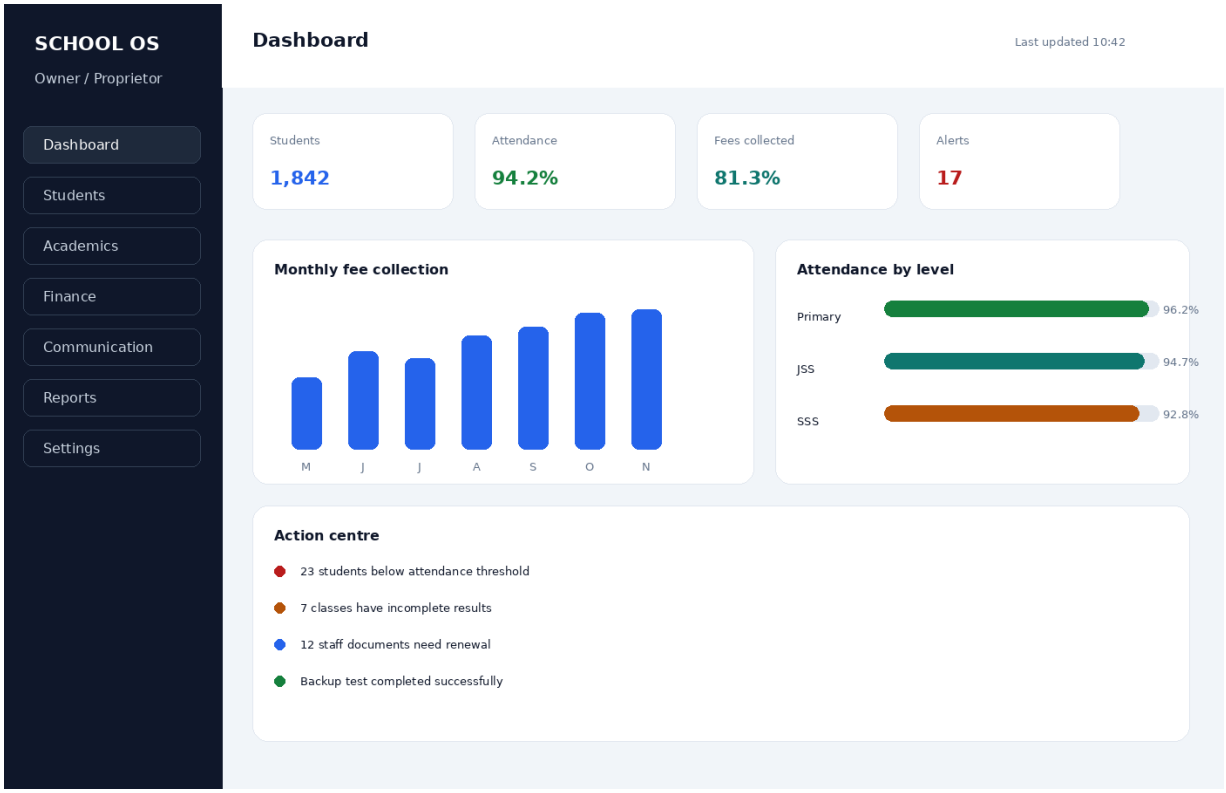


Figure 4. Illustrative owner dashboard visual with KPI cards, charts and an action centre. Numbers are mock data.

- Enrollment and movement
- Fee collection
- Outstanding balances
- Attendance
- Academic performance
- Admissions pipeline
- Staffing indicators
- Open issues
- System health summary

Principal dashboard

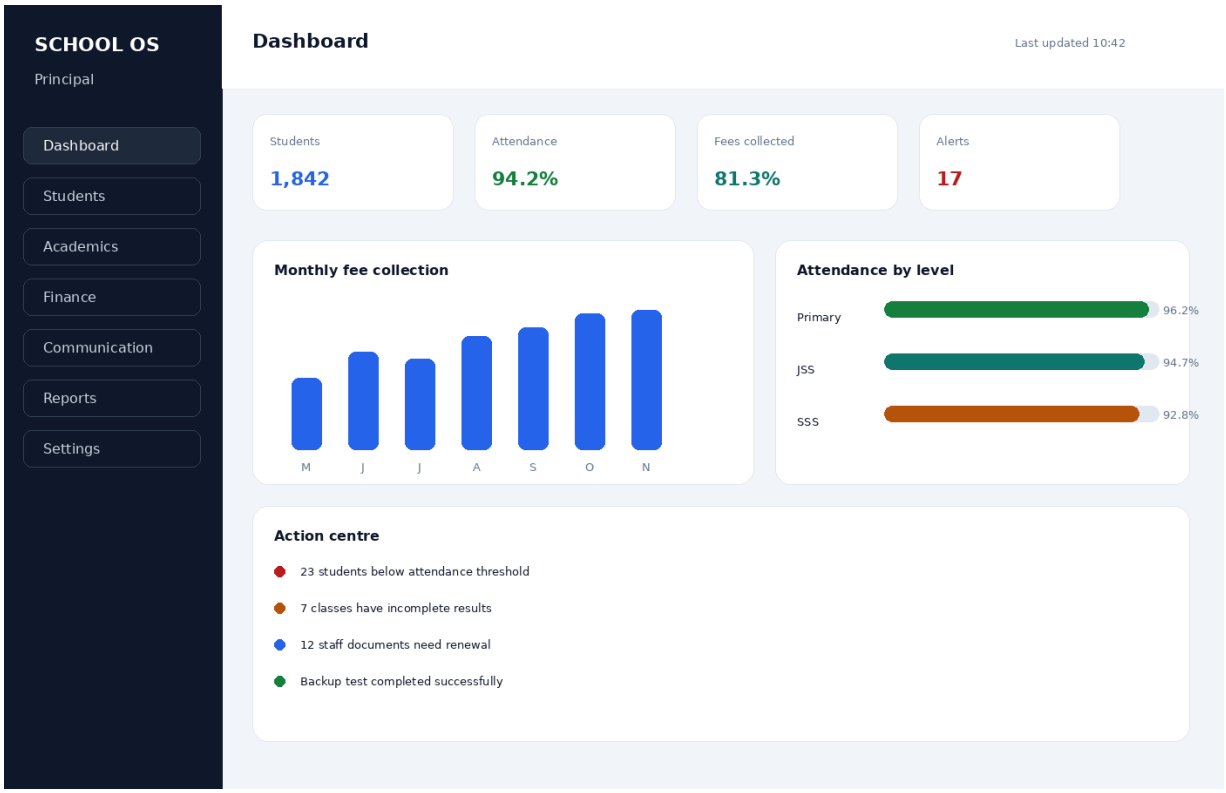


Figure 5. Illustrative principal dashboard; actual widgets should be based on school responsibilities. Numbers are mock data.

- Today attendance
- Classes not marked
- Pending approvals
- Incomplete results
- Student welfare alerts
- Teacher activity
- Upcoming events
- Important notices

IT / security dashboard

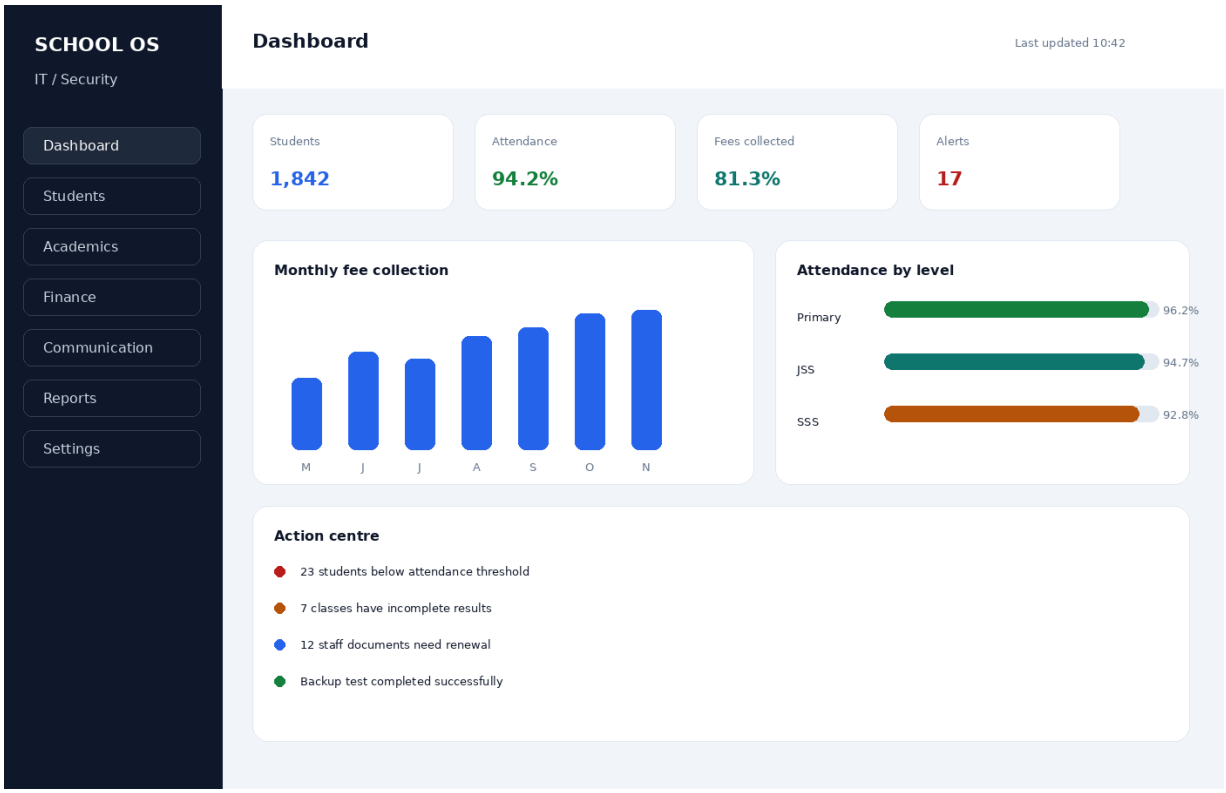


Figure 6. Illustrative IT/security dashboard; numbers are mock data.

- System uptime
- Backup status
- Last restore test
- Failed logins
- Privileged actions
- Pending access reviews
- Integration health
- Error rate
- Open incidents

Visual design requirements

Requirement	Implementation rule
KPI cards	Show value, comparison period if meaningful, timestamp and clickable drill-down.
Charts	Always label axis, unit and period. Avoid decorative charts.
Filters	Users can filter by campus, class, session, term, date and other authorized dimensions.
Exceptions	Alerts use thresholds and link directly to the relevant record or queue.
Exports	Provide authorized CSV/PDF exports with timestamp and filter context.
Freshness	Display data freshness or last updated time where real-time is not guaranteed.

Accessibility: do not communicate meaning through color alone; use labels, icons or text as well. Permissions must still be enforced server-side for every chart and drill-down.

21. Approval workflows and separation of duties

This is a core requirement from your brief. The platform must support actions that cannot be completed by one person alone. The design pattern is maker-checker, with a configurable third approver for high-risk actions.

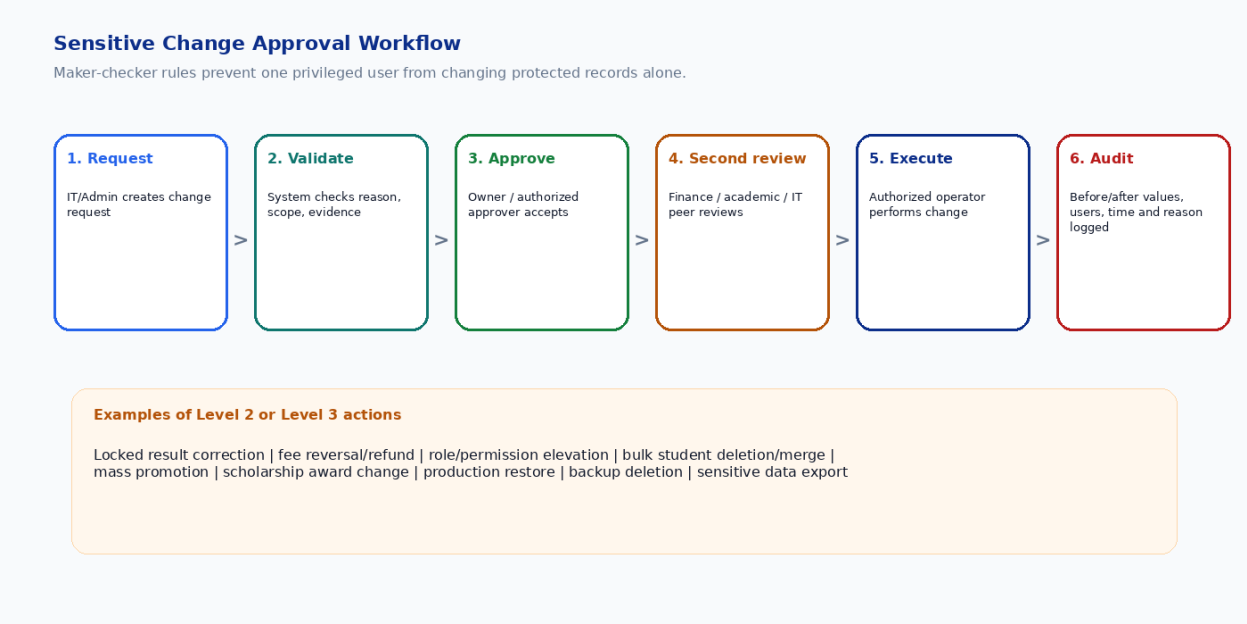


Figure 7. Sensitive change workflow.

Approval levels

Level	Meaning	Examples
0 - Direct	Low-risk action; no approval required	Mark attendance, draft announcement
1 - Single approval	Request by one user, approval by authorized supervisor	Edit normal student contact field
2 - Dual control	Two distinct people participate	Locked result correction, payment reversal
3 - Quorum / triple control	Three distinct roles or two approvers plus execution role	Production restore, bulk sensitive data deletion, privileged access recovery

Separation-of-duties rules

- The requester cannot approve their own request.
- The executor cannot be the sole approver of a protected change.
- Approvers must possess an explicit permission and scope.
- Approval can expire; stale requests should be revalidated.
- Changes to approval policy itself require elevated authorization.
- Emergency break-glass access must have mandatory post-event review.

Approval state machine

DRAFT -> SUBMITTED -> PENDING_APPROVAL -> APPROVED -> EXECUTING -> COMPLETED. Exception states: REJECTED, EXPIRED, CANCELLED, FAILED, ROLLED_BACK. Every transition should produce an audit event.

22. Security architecture

A school platform is a high-value target because it combines identity, children's records, financial transactions, documents and communications. The security architecture should use a recognized application-security baseline and then add school-specific controls.

Recommended security baseline

Use OWASP ASVS 5.0 as a verification-oriented baseline for application security. ASVS covers areas including input validation, business logic, web frontend security, APIs, file handling, authentication, sessions and authorization. [R3]

Authentication

- Secure password hashing
- MFA for privileged accounts
- Session rotation / revocation
- Rate limiting
- Suspicious login detection
- Recovery controls
- Device/session visibility
- Email/phone verification

Authorization

- RBAC
- Resource-level authorization
- Tenant isolation
- Scoped assignments
- Deny-by-default API policy
- Authorization tests for every protected endpoint

Encryption and secrets

- TLS in transit
- Encryption at rest where supported
- Secrets manager / protected environment variables
- Key rotation procedures
- Never log secrets
- Never store passwords reversibly

Application security

- Input validation
- Output encoding

- CSRF protection where applicable
- Secure cookies
- Security headers
- Content Security Policy where appropriate
- SQL injection defenses
- SSRF protections
- Safe file uploads
- Rate limiting
- Dependency scanning

Audit and security logs

OWASP ASVS 5.0 specifically includes verification requirements for logging authentication operations and failed authorization attempts. Use structured security events with useful metadata while avoiding logging the underlying sensitive data itself. [R4]

Security testing

- Unit and integration authorization tests
- Tenant isolation tests
- API fuzzing / negative tests
- Dependency scanning
- Static analysis
- Dynamic web testing
- File-upload tests
- Rate-limit tests
- Privilege escalation tests
- Backup/restore security tests
- Annual independent penetration test where commercially appropriate

23. Privacy, data governance and Nigerian compliance

The Nigeria Data Protection Act 2023 is the central Nigerian privacy statute relevant to this platform, and the NDPC provides current guidance and services around data protection. The product should be designed so compliance activities are supported by the system rather than documented only in a PDF policy. [R1][R2]

Data governance capabilities

- Data inventory
- Purpose / processing records
- Role and access mapping
- Retention configuration
- Deletion requests
- Correction requests
- Export / portability support
- Incident / breach workflow
- Vendor/subprocessor register
- Privacy notice versioning
- Consent/guardian mechanisms where applicable

Children and sensitive data

Because the platform may process children's records and potentially health information, the school and platform operator should explicitly identify the categories of data, the lawful basis and the safeguards. Do not assume that one generic consent checkbox solves all education-related processing. The actual contractual and legal position should be reviewed by appropriate Nigerian privacy counsel or a qualified privacy professional.

Likely controller / processor model

In a typical SaaS arrangement, the school may determine many purposes and means of processing student and family data while the SaaS operator processes data to provide the service. The exact controller/processor allocation depends on the contract and actual processing, so the product should support a clear data-processing agreement and documented responsibilities instead of assuming the legal role automatically.

Data subject operations

The NDPC describes rights including information, access, rectification, objection, restriction, portability and erasure. The product should make it technically possible to locate, export, correct and appropriately delete data, subject to lawful retention requirements and the school policy. [R2]

24. Backup, disaster recovery and business continuity

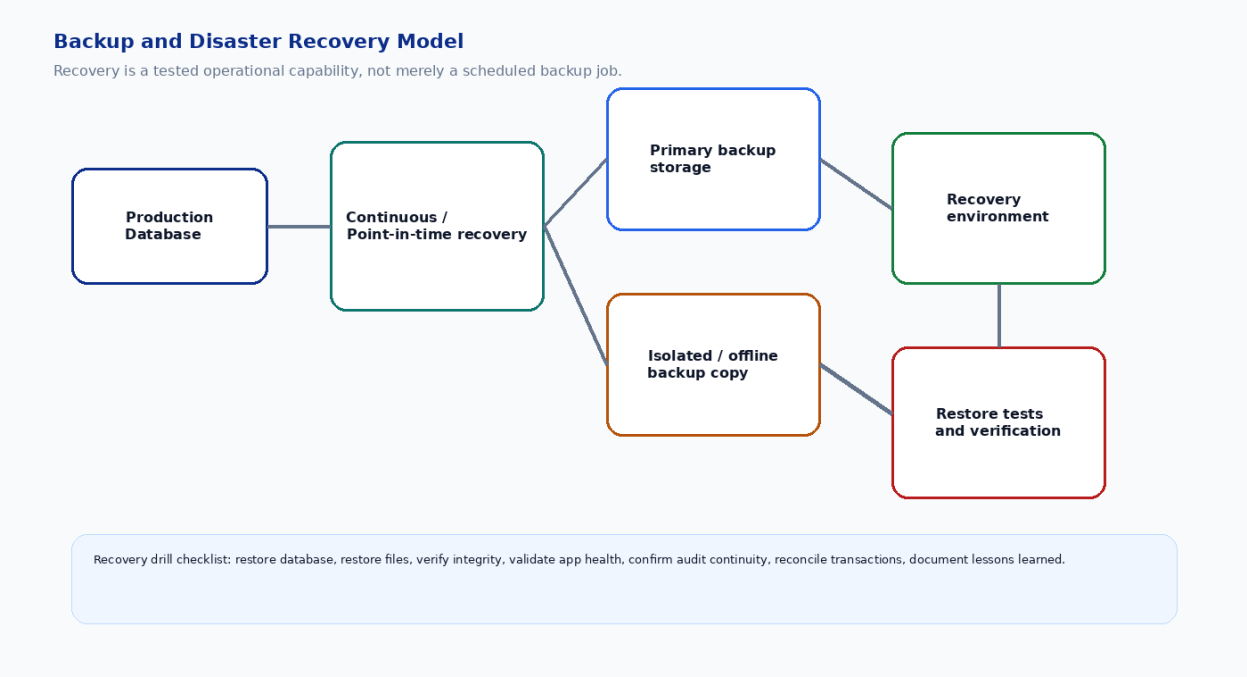


Figure 8. Backup and recovery model. The exact infrastructure should be selected after capacity and threat analysis.

Backup principles

- Multiple copies
- Different storage or failure domains
- Encrypted backups
- Separate credentials where possible
- Immutable or protected copies where appropriate
- Retention policy
- Backup monitoring
- Restore testing
- Documented recovery runbook

CISA recommends maintaining offline, encrypted backups of critical data and regularly testing their availability and integrity. NIST CSF 2.0 likewise calls for verifying backup integrity before restoration and verifying restored assets after recovery. [R6][R7]

RPO and RTO

Metric	Target for planning	Meaning
RPO	Example: 15 minutes	Approximate maximum acceptable data loss window for a severe failure.
RTO	Example: 60-120 minutes	Approximate target to restore the service after a major incident.

Recovery targets are engineering planning assumptions until infrastructure and drills prove them. The school contract should state the actual service commitment separately.

Recovery runbook

111. Declare incident
112. Freeze unsafe changes
113. Assess scope
114. Provision recovery environment
115. Restore database / point-in-time state
116. Restore files
117. Verify integrity
118. Run application health checks
119. Validate audit continuity
120. Reconcile financial transactions
121. Confirm identity and permission state
122. Switch service traffic
123. Notify stakeholders
124. Document lessons learned
125. Schedule follow-up remediation

Backup deletion protection

The ability to delete all backups should never sit with a normal school IT administrator. Prefer isolated backup credentials and platform-level controls. If a destructive backup operation exists, require elevated authorization and a clear audit trail.

25. Scalability and multi-tenant architecture

Tenant model

The platform should use logical tenant isolation so that every school is represented as an organizational boundary. Every tenant-aware query and mutation must verify tenant scope server-side.

Scalability approach

- Start with a modular monolith to reduce operational overhead
- Keep domain boundaries explicit
- Use background queues for heavy tasks
- Use object storage for large files
- Cache carefully; never let cache become an ungoverned source of truth
- Use read models for expensive dashboards where justified
- Add indexes based on measured query patterns
- Introduce read replicas when required
- Partition high-volume event tables if evidence supports it
- Split services only when scale or ownership boundaries justify it

Tenant isolation strategies

Strategy	Pros	Trade-offs
Shared DB, tenant key	Simple and cost-efficient	Requires rigorous authorization and query discipline
Schema per tenant	Stronger logical separation	Higher operational complexity at scale
DB per tenant	Strong isolation	Higher cost and operational overhead

A practical SaaS can begin with shared infrastructure and strict tenant-aware authorization, then offer stronger isolation tiers for enterprise customers when justified.

26. Data model and database design

Core domains

- Tenant / school
- Identity / users
- Roles / permissions
- Students / guardians
- Admissions
- Enrollment / class membership
- Academics
- Attendance
- Assessments / results
- Finance
- Communication
- Operations
- LMS
- Audit / security
- Subscriptions / billing

Representative entities

Domain	Representative entities
Identity	User, Role, Permission, Membership, Session, MFAFactor
School	School, Campus, AcademicSession, Term, Class, ClassArm, Subject
Student	Student, Guardian, GuardianLink, Enrollment, Document
Academics	Assignment, Assessment, Exam, Score, GradeScheme, Result, ReportCard
Finance	FeeStructure, Charge, Invoice, InvoiceItem, Payment, Refund, Discount, Scholarship
Messaging	Notification, Template, MessageThread, Message, DeliveryAttempt
Governance	ApprovalRequest, ApprovalStep, AuditEvent, BreakGlassSession
Operations	Book, Loan, Vehicle, Route, HostelRoom, Asset, MaintenanceRecord
LMS	Course, Module, Lesson, Resource, Quiz, Submission, Progress

Ledger rule

Financial records should behave like a ledger, not like mutable page fields. A balance can be derived from authoritative charges and settled payments; if a reversal occurs, record a new compensating transaction and approval rather than silently editing history.

Immutable audit records

Audit events should be append-only from the application perspective. The application may write them; normal end users should not edit or delete them. Retention and archival can be implemented operationally without pretending the past did not happen.

27. API, integrations and background jobs

API principles

- Versioned contracts
- Server-side authorization
- Consistent error format
- Idempotency keys for retriable financial operations
- Pagination
- Filtering
- Rate limits
- Request correlation IDs
- Audit-event generation
- OpenAPI documentation

Integration boundaries

Integration	Boundary rule
Payments	Verify provider callbacks; provider is the payment rail, platform is the school ledger.
Email/SMS/WhatsApp	Provider sends; platform records intent/status and retries.
Zoom/Meet	Provider hosts live session; platform owns course/lesson relationship.
Storage	Object store holds private files; app controls authorization and signed access.
Identity provider	External SSO may authenticate, but local authorization still enforces school scope.

Background workers

- Invoice generation
- Notification delivery
- PDF report generation
- Analytics aggregation
- File scanning
- Backup verification jobs
- AI tasks
- Scheduled reminders
- Data import processing
- Webhook reconciliation

28. Development standards and testing

Engineering standards

- TypeScript strict mode where applicable
- Domain-oriented folder structure
- Validated environment configuration
- Database migrations under version control
- No direct production edits
- Code review for protected modules
- Automated linting and formatting
- Dependency pinning / lockfile
- Structured logging
- Typed API contracts

Testing pyramid

Level	Examples
Unit	Fee calculation, grade calculation, permission predicates
Integration	Payment reconciliation, admission conversion, approval transitions
API	Authentication, authorization, tenant isolation, validation
E2E	Parent payment -> receipt -> school ledger; result entry -> approval -> publish
Security	Privilege escalation, IDOR, injection, file upload, session abuse
Performance	Dashboard queries, report generation, concurrent imports
Recovery	Backup restore, migration rollback, incident runbook

Release evidence

Every release should have a compact evidence package: change summary, linked requirement, tests executed, migration notes, security impact, rollback plan and monitoring notes. This is especially important when using an autonomous coding platform.

29. Antigravity development workflow

Google Antigravity is an agentic development platform that can operate across the editor, terminal and browser and can orchestrate agents/subagents and artifacts. Its own documentation emphasizes planning, execution, verification and artifacts. That makes it suitable for this project when used as an engineering system rather than as a "build everything" button. [R8][R9]

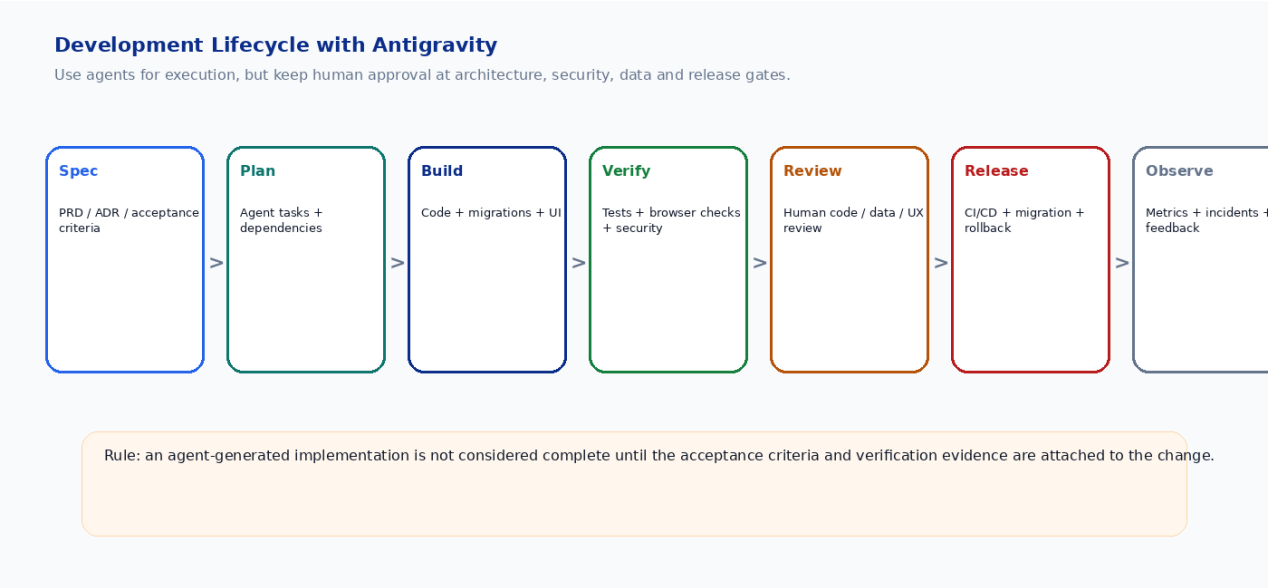


Figure 9. Recommended Antigravity workflow for this project.

Workspace structure

Path	Purpose
/docs/product	PRD, personas, workflows, acceptance criteria
/docs/architecture	ADRs, diagrams, threat model, data flow
/docs/security	ASVS checklist, security tests, incident procedures
/docs/runbooks	Deployment, rollback, backup, restore, handover
/apps/web	Next.js web application
/apps/api	Backend API / domain logic
/packages/db	Schema, migrations, data access
/packages/ui	Shared UI components
/workers	Background processing
/infra	Infrastructure as code / environment configuration
/tests	Cross-domain fixtures, E2E and security suites

Agent instructions

- One source of truth for requirements
- Never invent a missing business rule; flag it as a gap
- Never bypass tests because an implementation looks obvious
- Never modify protected data models without an ADR
- Never use production secrets in development
- Always show changed files and verification evidence

- Never claim a feature is complete without the acceptance criteria passing
- Keep migrations reversible where feasible

Suggested Antigravity artifacts

- Product spec
- Architecture decision record
- Threat model
- Database ERD
- API contract
- UI flow
- Test plan
- Implementation plan
- Verification report
- Release note
- Migration runbook

30. Step-by-step development roadmap

The goal is a complete commercial product, but implementation should be sequential. Each phase ends with a demonstrable, testable capability. Do not move to the next phase merely because the code exists.

Phase 0 - Discovery and requirements

- School interviews
- Process maps
- Roles
- Feature inventory
- Non-functional requirements
- Acceptance criteria
- Pilot scope

Phase 1 - Product foundation

- Repo/monorepo
- CI/CD
- Environment model
- Design system
- Architecture records
- Base database
- Observability scaffolding

Phase 2 - Identity and tenancy

- Schools
- Users
- Roles
- Permissions
- Tenant isolation
- MFA for privileged roles
- Audit events

Phase 3 - School setup

- Campuses
- Sessions
- Terms
- Classes
- Subjects
- Configuration
- Branding

Phase 4 - Student and admissions

- Applications
- Documents
- Guardians
- Student record
- Enrollment
- Imports

Phase 5 - Academics

- Subjects
- Timetable
- Attendance
- Assignments
- Assessments
- Results

Phase 6 - Finance

- Fee structures
- Invoices
- Payments
- Receipts
- Reconciliation
- Refund/reversal approvals

Phase 7 - Communication

- Notifications
- Announcements
- Messaging
- Email/SMS/WhatsApp adapters

Phase 8 - Dashboards and reports

- Role dashboards
- Exports
- Report cards
- Analytics
- Alerts

Phase 9 - Operations

- Library
- Transport

- Hostel
- Clinic
- Inventory
- Events
- PTA

Phase 10 - Scholarships

- Scholarship programmes
- Vouchers
- Sponsors
- Awards

Phase 11 - LMS

- Courses
- Lessons
- Resources
- Quizzes
- Assignments
- Progress
- External live-class integration

Phase 12 - AI / automation

- Workflow engine
- AI assistants
- Scheduled reports
- Performance summaries
- Exception detection

Phase 13 - Hardening

- Security review
- Performance
- Recovery drills
- Privacy review
- Accessibility
- Load tests
- Penetration testing

Phase 14 - Pilot and handover

- Data migration
- Training
- Parallel run

- Go-live
- Acceptance
- IT handover
- Support launch

Phase exit rule

A phase exits only when its acceptance criteria, tests, documentation and operational readiness are satisfied. The agent can produce implementation artifacts; a human must still review data, security, business rules and release readiness.

31. Deployment, release and operations

Environment model

Environment	Purpose
Local	Developer work; synthetic or sanitized data
Development	Integrated feature testing
Staging	Release candidate and migration rehearsal
Production	Real school data; protected deployment controls

Deployment pipeline

126. Open change request
127. Run unit/integration tests
128. Run security checks
129. Build artifacts
130. Apply staging migration
131. Run E2E and smoke tests
132. Human release approval
133. Backup / pre-release check where required
134. Deploy production
135. Run health checks
136. Monitor
137. Close release with evidence

Rollback

Rollback must distinguish application code rollback from database rollback. A database migration may be irreversible, so prefer forward-compatible migrations and explicit remediation scripts. Maintain a tested application rollback path and a clear data recovery plan.

32. School implementation and data migration

Implementation stages

138. Discovery meeting
139. Data audit
140. Configuration design
141. Migration template delivery
142. Data cleanup
143. Dry-run import
144. Reconciliation
145. Training
146. Pilot / parallel run
147. Go-live
148. Hypercare
149. Acceptance sign-off

Migration controls

- Source file inventory
- Column mapping
- Required field validation
- Duplicate detection
- Reference-data mapping
- Historical result handling
- Historical payment handling
- Document mapping
- Import report
- Error quarantine
- Rollback / re-import strategy
- Final reconciliation

Do not import dirty data blindly

If the source school records contain duplicate students or inconsistent guardians, the implementation process should surface those issues for school approval. The platform should not silently decide which historical record is correct.

33. School IT handover model

The handover must make the school operationally independent enough to manage routine administration without granting unsafe privileges. The school should know what it owns, what the SaaS provider operates, what requires approval and how to recover from incidents.

Ownership model

Item	Recommended ownership / control
School data	School-controlled contractual data, subject to legal obligations and retention rules.
Application IP	Depends on commercial agreement; SaaS default is vendor-owned platform IP.
School domain	Prefer school ownership; platform receives delegated DNS access as required.
Payment merchant account	Prefer school-owned merchant account where commercially and regulatorily appropriate.
Backup copies	Platform-operated backups for SaaS service, with contractual access/recovery obligations; define retention and exit handling.
User accounts	School controls operational user lifecycle; platform controls platform-level accounts and support access.
Infrastructure secrets	Managed by platform operator; not handed to ordinary school staff.
Emergency access	Controlled, time-limited and audited.

Handover package

- System overview
- School-specific configuration document
- User and role matrix
- Approval matrix
- Data ownership and exit terms
- Data export procedure
- Admin guide
- Teacher guide
- Parent onboarding guide
- IT operations guide
- Backup / recovery responsibilities
- Incident escalation path
- Integration inventory
- Domain/DNS ownership map
- Support contacts
- Release policy
- Known limitations
- Acceptance certificate

School IT privileges

Capability	School IT	School owner	Vendor / platform operator
Create routine user	Yes, within role scope	Can approve policy	Support / platform access
Change protected student identity	Request only	Approve	Technical support if needed
Correct locked result	Request	Approve	Assist only
Refund payment	Request	Approve	Provider/platform reconciliation as applicable
Change privileged role	Request	Approve	Platform may enforce policy
Production database access	No by default	No by default	Restricted platform operations
Restore production	Request	Approve / authorize	Execute under runbook
Delete backups	No	No	Highly restricted with separate controls

Training and sign-off

150. Role-based training
151. Scenario exercises
152. Approval workflow training
153. Incident simulation
154. Data export simulation
155. Backup/recovery explanation
156. Administrator competency check
157. School sign-off

34. Support, maintenance and service operations

Support levels

Priority	Example	Response model
P1 Critical	Service unavailable, major payment issue, suspected breach	Immediate incident handling per contract
P2 High	Core workflow broken for a school	Priority support
P3 Normal	Feature defect with workaround	Standard queue
P4 Request	Training/configuration/question	Scheduled support

Maintenance

- Security patches
- Dependency updates
- Database maintenance
- Performance review
- Backup tests
- Restore drills
- User access reviews
- Integration credential rotation
- Documentation refresh
- Quarterly recovery exercise

School success metrics

- Active users
- Parent activation
- Digital fee payment usage
- Attendance completeness
- Result completion time
- Support tickets per 100 users
- Feature adoption
- Failed transaction rate
- Time to resolve incidents

35. Pilot validation and product acceptance

Pilot structure

158. Select one or two schools with a cooperative leadership team
159. Define pilot scope and exclusions
160. Define success measures
161. Collect sanitized sample data
162. Configure school
163. Train key users
164. Run controlled pilot
165. Measure actual workflow impact
166. Collect structured feedback
167. Fix high-impact gaps
168. Perform acceptance test
169. Decide on commercial rollout

Suggested pilot success criteria

Area	Example criterion
Attendance	95%+ of expected attendance registers completed by deadline during pilot period.
Finance	Online payment records reconcile correctly against provider confirmations.
Results	No published result changes bypass the configured approval workflow.
Parent adoption	Target percentage of invited parents successfully activate accounts.
Support	Critical issues have documented resolution and recurrence controls.
Recovery	Backup restore drill completes and application integrity is verified.

Use real success criteria, not vanity metrics

A school using the dashboard twice does not prove product value. Measure the workflows you intend to replace or improve: reconciliation, result processing, communication, fee collection operations and administrative visibility.

36. Commercial packaging and school proposal strategy

Proposal structure

170. Current-state discovery
171. Problems observed
172. Proposed SchoolOS model
173. Role-specific benefits
174. Security and privacy model
175. Implementation plan
176. Data migration approach
177. Training and handover
178. Support model
179. Pilot scope
180. Commercial terms
181. Acceptance criteria

Demo sequence

182. Login as school owner
183. Show dashboard
184. Open a student
185. Show academic record
186. Mark attendance
187. Create fee invoice
188. Make a simulated payment
189. Show reconciliation
190. Approve a sensitive correction
191. Publish a result
192. Switch to parent portal
193. Show parent notification
194. Open school analytics
195. Show IT/security controls
196. End with recovery and handover model

Positioning statement

Suggested positioning

A secure, scalable digital operating system for schools that connects administration, academics, finance, parents, communication and learning in one controlled platform - with approvals, dashboards, audit trails and recovery built into the foundation.

Contract topics to define

- Data ownership
- Permitted processing
- Subprocessors
- Service availability
- Support hours
- Incident communication
- Backup and recovery
- Data export on exit
- Deletion/retention
- Payment responsibilities
- Custom development
- Training
- Change management
- Security review
- Liability and insurance where applicable

37. KPI framework

School KPIs

- Enrollment
- Attendance rate
- Fee collection rate
- Outstanding fees
- Result completion
- Pass rate
- Assignment completion
- Parent activation
- Communication delivery
- Incident count

Product KPIs

- Active schools
- Active users
- Parent activation
- Retention
- Support tickets
- Feature adoption
- Payment transaction success
- Notification delivery
- System uptime
- API error rate
- Backup success rate
- Restore drill success
- Mean time to recovery

Engineering KPIs

- Deployment frequency
- Change failure rate
- Lead time for changes
- Test coverage by critical domain
- Open security findings
- Mean time to patch critical issue
- Database query performance
- Queue latency

38. Risk register and mitigation

Risk	Impact	Mitigation
Feature sprawl	High	Use domain boundaries and phase gates; do not build every module at once.
Poor school data quality	High	Pre-migration audit, reconciliation and approval.
Unauthorized privileged action	Critical	RBAC, dual control, audit logs, MFA and alerts.
Payment mismatch	Critical	Provider verification, idempotency, reconciliation and exception queue.
Data loss	Critical	Multiple backups, isolated copy, restore tests, RPO/RTO.
Vendor dependency	Medium	Adapter interfaces and exportable school records.
Internet instability	Medium	PWA/offline-tolerant workflows for selected functions.
AI hallucination	High	Human review, retrieval, evidence and limited write access.
School adoption failure	High	Role-based training, simple UX, pilot and parallel run.
Single-person knowledge risk	High	Documentation, handover, runbooks and shared repository.
Security breach	Critical	Secure SDLC, monitoring, incident response, testing and least privilege.
Regulatory misunderstanding	High	Professional legal/privacy review and ongoing compliance updates.

39. Master screen/page inventory

Platform portal

- Login
- MFA
- Platform dashboard
- Schools
- School detail
- Subscriptions
- Billing
- Usage
- Support
- System health
- Security events
- Audit logs
- Backups
- Incidents
- Feature flags
- Integrations

School administration

- Dashboard
- Students
- Parents
- Teachers
- Classes
- Subjects
- Academic sessions
- Terms
- Campuses
- Admissions
- Documents
- Calendar
- Announcements
- Settings

Finance

- Finance dashboard
- Fee structures
- Invoices
- Payments

- Reconciliation
- Receipts
- Refunds
- Reversals
- Discounts
- Scholarships
- Expenses
- Reports

Academics

- Academic dashboard
- Timetable
- Attendance
- Assignments
- Assessments
- Exams
- Results
- Approvals
- Report cards
- Promotion
- Academic analytics

Operations

- Library
- Transport
- Hostel
- Clinic
- Cafeteria
- Inventory
- Assets
- Maintenance
- Events
- PTA

LMS

- Courses
- Course editor
- Lesson editor
- Resources
- Quizzes
- Assignments
- Submissions

- Progress
- Certificates
- Live class settings

Family/student

- Family dashboard
- Child view
- Payments
- Attendance
- Results
- Assignments
- Learning
- Messages
- Events
- Documents
- Requests
- Profile

40. Approval matrix

Action	Requester	Approver 1	Approver 2	Executor	Audit
Edit locked result	Teacher/admin	Academic lead	Principal / owner delegate	Authorized admin	Required
Reverse payment	Bursar	Finance lead	Owner / principal delegate	Finance operator	Required
Change privileged role	IT admin	School owner	Platform admin where required	IT admin	Required
Bulk student deletion/merge	Registrar/IT	School owner	Second authorized officer	Platform operator / IT	Required
Production restore	IT / vendor	School owner / incident lead	Platform operations	Platform operations	Required
Delete backup	Platform operations	Security/owner role	Second platform approver	Restricted operator	Required
Sensitive data export	IT/admin	School owner	Privacy-authorized reviewer where applicable	Authorized exporter	Required
Change fee structure after publication	Bursar	Finance lead	Owner/principal	Bursar	Required
Change grading scheme after result entry	Academic lead	Principal	Owner/delegate	Academic admin	Required

Approval UX requirements

- Show reason field
- Show target record
- Show before/after values
- Show risk level
- Show approver identity
- Require re-authentication for highest-risk actions
- Prevent self-approval
- Show expiry
- Show comments
- Store immutable audit event

41. Definition of done and release checklist

Feature definition of done

- Requirement linked
- Acceptance criteria passed
- UI verified
- API verified
- Authorization tested
- Tenant isolation tested
- Unit/integration/E2E tests as appropriate
- Audit events implemented
- Error states handled
- Loading states handled
- Empty states handled
- Accessibility reviewed
- Documentation updated
- Monitoring added where needed
- Rollback/recovery considered

Production release checklist

197. Change reviewed
198. Tests green
199. Security checks green
200. Migration reviewed
201. Backup status healthy
202. Rollback plan ready
203. Environment variables verified
204. Third-party provider status checked
205. Release approved
206. Deployment executed
207. Smoke tests passed
208. Monitoring watched
209. Release notes published
210. Support informed

42. Reference architecture summary

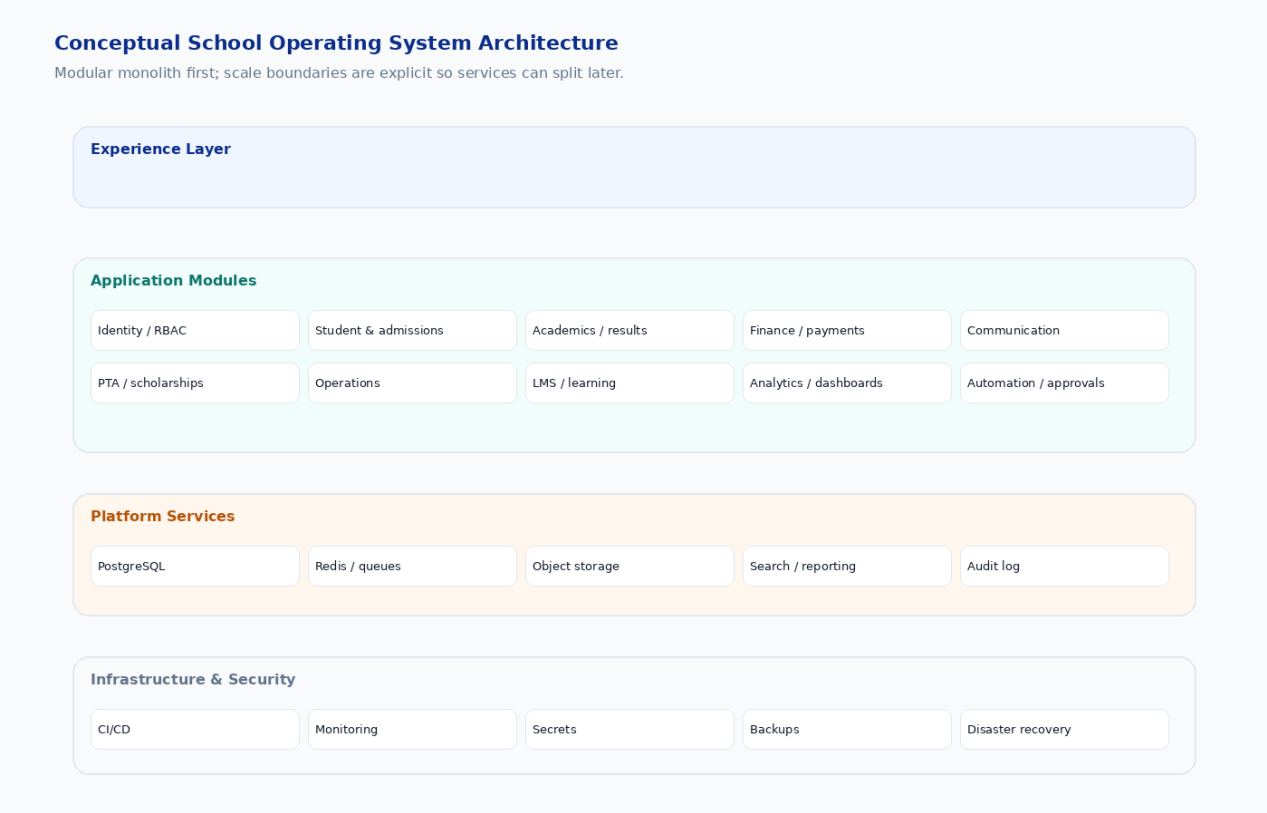


Figure 10. Reference architecture summary.

Recommended starting stack

Concern	Suggested starting choice	Reason
Web	Next.js + TypeScript	Strong full-stack web foundation and familiar React model.
Backend	Structured Node.js/TypeScript API	Good fit for shared types and a developer working in one ecosystem.
Database	PostgreSQL	Strong relational model for school data, transactions and reporting.
Cache/queues	Redis + queue worker	Supports rate limits, background jobs and asynchronous workflows.
Storage	Private object storage	Better for documents and report files than database blobs.
CI/CD	GitHub Actions or equivalent	Automated verification and release workflow.
Observability	Managed logs/metrics/traces	Detect problems before schools do.
Payments	Provider such as Paystack/Flutterwave	Avoid owning card-processing infrastructure.
Video	Zoom/Google Meet integration initially	Do not build a conferencing platform before product-market need.

43. Reference sources and compliance notes

The following sources were checked for the current blueprint and should be reviewed again before production launch because regulatory and vendor requirements can change.

[R1] Nigeria Data Protection Commission. Nigeria Data Protection Act, 2023. Official NDPC resource: https://ndpc.gov.ng/download/nigeria-data-protection-act-2023
Used for the Nigerian data-protection framework, definitions and data-subject rights context.

[R2] Nigeria Data Protection Commission. NDP Act 2023 - General Application and Implementation Directive (GAID) 2025. Official PDF: https://ndpc.gov.ng/wp-content/uploads/2025/03/NDP-ACT-GAID-2025-MARCH-20TH.pdf
Used for current implementation guidance, including compliance organization and controller/processor context.

[R3] OWASP Foundation. Application Security Verification Standard (ASVS) 5.0. Official project page: https://owasp.org/projects/asvs
Used as the application-security verification baseline.

[R4] OWASP ASVS 5.0, Security Events section. https://cornucopia.owasp.org/taxonomy/asvs-5.0/16-security-logging-and-error-handling/03-security-events
Used for authentication and authorization security-event logging requirements.

[R5] PCI Security Standards Council. PCI DSS v4.0.1. https://blog.pcisecuritystandards.org/just-published-pci-dss-v4-0-1
Used for the payment-security discussion and the recommendation to minimize direct handling of card data.

[R6] CISA. #StopRansomware Guide. https://www.cisa.gov/stopransomware/ransomware-guide
Used for offline/isolated backup and backup-testing guidance.

[R7] NIST. Cybersecurity Framework 2.0. https://www.nist.gov/publications/nist-cybersecurity-framework-csf-20
Used for governance, protection, detection, response and recovery framing.

[R8] Google Antigravity. Antigravity 2.0 overview. https://www.antigravity.google/docs/overview
Used for current Antigravity capabilities and agent/artifact workflow framing.

[R9] Google Antigravity IDE overview. https://www.antigravity.google/docs/ide/overview/
Used for editor, browser, agents, artifacts and verification workflow framing.

[R10] Edves. School management solution page. https://www.edves.com/solutions/school-management/
Used only as vendor-stated market evidence for existing Nigerian school-management category breadth.

[R11] SchoolTry. About / product pages. https://schooltry.com/
Used only as vendor-stated market evidence for school-management, virtual classroom and LMS breadth.

[R12] NoviaSMS. Product page. https://noviasms.com/
Used only as vendor-stated evidence that sensitive changes can be placed behind second-person approval workflows.

Legal and security review before launch

Before signing schools, have the actual SaaS agreement, privacy/data-processing terms, payment arrangement, retention policy, incident plan and security architecture reviewed by the appropriate Nigerian legal/privacy and security professionals. The software can encode controls, but contractual and regulatory compliance is broader than code.

END OF BLUEPRINT